

RESUMEN COMPLETO

Sistemas Cognitivos Artificiales

Maestría en Inteligencia Artificial

Manual Completo - 11 Temas

TEMA 1: INTRODUCCIÓN AL APRENDIZAJE PROFUNDO

1.1 ¿Qué es Deep Learning?

Deep Learning es un subcampo del Machine Learning que permite a las computadoras aprender representaciones de datos con múltiples niveles de abstracción. Utiliza redes neuronales artificiales con arquitecturas profundas (múltiples capas) para modelar patrones complejos en grandes cantidades de datos.

1.2 Factores Clave del Éxito del Deep Learning

- **Mayor disponibilidad de datos:** Internet, redes sociales, sensores IoT generan cantidades masivas de datos
- **Mejora de la capacidad de cómputo:** GPUs permiten entrenar redes profundas en tiempo razonable
- **Mejoras algorítmicas:** Dropout, Batch Normalization, nuevas funciones de activación

1.3 Aplicaciones Exitosas

- **Visión por computador:** Reconocimiento facial, detección de objetos, segmentación de imágenes
- **Procesamiento de lenguaje natural:** Traducción automática, análisis de sentimientos, chatbots
- **Reconocimiento del habla:** Asistentes virtuales (Siri, Alexa, Google Assistant)
- **Aprendizaje por refuerzo:** AlphaGo venció al campeón mundial de Go

1.4 Redes Neuronales - Conceptos Fundamentales

- **Neurona artificial:** Unidad básica que recibe inputs, aplica pesos, suma, agrega bias y aplica función de activación
- **Pesos (weights):** Parámetros que se aprenden durante el entrenamiento
- **Bias:** Término independiente que permite desplazar la función de activación
- **Función de activación:** Introduce no-linealidad (sigmoid, tanh, ReLU)
- **Fully Connected Networks:** Cada neurona de una capa conectada a todas las de la siguiente

1.5 Inspiración Biológica

Las redes neuronales artificiales se inspiran en el cerebro humano, aunque con diferencias significativas. Las neuronas biológicas reciben señales eléctricas (sinapsis), las procesan y generan una respuesta. De manera análoga, las neuronas artificiales reciben inputs numéricos, los ponderan, y producen un output mediante una función de activación.

TEMA 2: ENTRENAMIENTO DE REDES NEURONALES

2.1 Proceso de Entrenamiento

El entrenamiento de una red neuronal implica ajustar los pesos y biases para minimizar una función de pérdida (loss function) que mide la diferencia entre las predicciones y los valores reales.

2.2 Backpropagation (Retropropagación)

- Algoritmo fundamental para calcular gradientes de la función de pérdida respecto a cada peso
- Usa la regla de la cadena para propagar el error desde la salida hacia las capas anteriores
- Permite actualizar eficientemente todos los pesos de la red

2.3 Gradient Descent (Descenso del Gradiente)

- **Batch Gradient Descent:** Usa todo el dataset en cada iteración (lento para datasets grandes)
- **Stochastic Gradient Descent (SGD):** Usa un ejemplo a la vez (rápido pero ruidoso)
- **Mini-batch Gradient Descent:** Compromiso entre ambos, usa pequeños lotes (batch size típico: 32, 64, 128, 256)

2.4 Learning Rate (Tasa de Aprendizaje)

- Hiperparámetro MÁS IMPORTANTE del entrenamiento
- Controla el tamaño del paso en la actualización de pesos
- Learning rate muy alto → divergencia, saltos erráticos
- Learning rate muy bajo → convergencia muy lenta, puede quedar atrapado en mínimos locales
- Valores típicos: 0.001, 0.0001, 0.00001

2.5 Optimizadores Avanzados

- **Momentum:** Acumula velocidad en direcciones consistentes del gradiente, acelera convergencia
- **RMSprop:** Adapta learning rate por parámetro usando promedio móvil de gradientes al cuadrado
- **Adam (Adaptive Moment Estimation):** Combina Momentum y RMSprop. El MÁS USADO actualmente. Parámetros típicos: $\alpha=0.001$, $\beta_1=0.9$, $\beta_2=0.999$
- **AdaGrad:** Adapta learning rate basado en frecuencia de actualizaciones

2.6 Función de Pérdida (Loss Function)

- **Mean Squared Error (MSE):** Para regresión, penaliza errores grandes cuadráticamente
- **Cross-Entropy:** Para clasificación, mide divergencia entre distribuciones de probabilidad
- **Binary Cross-Entropy:** Clasificación binaria
- **Categorical Cross-Entropy:** Clasificación multiclase (una clase por ejemplo)

2.7 Regularización - Prevenir Overfitting

- **L1 Regularization (Lasso):** Penaliza suma de valores absolutos de pesos, induce sparsity

- **L2 Regularization (Ridge):** Penaliza suma de cuadrados de pesos, mantiene pesos pequeños
- **Dropout:** Durante entrenamiento, apaga aleatoriamente neuronas con probabilidad p (típicamente 0.5). Fuerza representaciones redundantes
- **Early Stopping:** Detener entrenamiento cuando el error en validación empieza a aumentar
- **Data Augmentation:** Aumentar datos artificialmente (rotaciones, flips, crops en imágenes)

2.8 División de Datos

- **Training set (70-80%):** Para entrenar el modelo
- **Validation set (10-15%):** Para ajustar hiperparámetros y prevenir overfitting
- **Test set (10-15%):** Para evaluación final, NUNCA usado durante entrenamiento

TEMA 3: FRAMEWORKS DE DEEP LEARNING

3.1 ¿Por qué usar Frameworks?

Implementar redes neuronales desde cero es complejo y propenso a errores. Los frameworks proveen APIs de alto nivel, cálculo automático de gradientes, optimización, y soporte para GPUs.

3.2 TensorFlow

- Desarrollado por Google Brain (2015)
- Grafo de computación estático (TensorFlow 1.x) → dinámico con Eager Execution (TensorFlow 2.x)
- Excelente ecosistema: TensorFlow Serving (producción), TensorFlow Lite (móviles), TensorFlow.js (web)
- Muy usado en producción por grandes empresas
- Curva de aprendizaje más empinada

3.3 Keras

- API de alto nivel, simple e intuitiva
- Originalmente independiente, ahora integrada oficialmente en TensorFlow (tf.keras)
- Ideal para prototipado rápido y principiantes
- Soporta backends: TensorFlow, Theano, CNTK

3.4 PyTorch

- Desarrollado por Facebook AI Research (2016)
- Grafos de computación DINÁMICOS (define-by-run) → más flexible
- Sintaxis pythonic, más intuitivo para programadores Python
- Muy popular en INVESTIGACIÓN académica
- Debugging más fácil que TensorFlow 1.x
- TorchScript para deployment en producción

3.5 Otros Frameworks

- **Caffe:** Especializado en visión por computador, muy rápido pero menos flexible
- **MXNet:** Adoptado por Amazon, eficiente y escalable
- **JAX:** De Google, para investigación avanzada con diferenciación automática

3.6 Comparación TensorFlow vs PyTorch

- **TensorFlow:** Mejor para producción, ecosistema maduro, deployment más fácil
- **PyTorch:** Mejor para investigación, más flexible, debugging más simple
- Ambos convergen en funcionalidades (TensorFlow 2.x es más pythonic, PyTorch mejora deployment)

TEMA 4: ASPECTOS PRÁCTICOS DEL ENTRENAMIENTO DE REDES PROFUNDAS

4.1 Funciones de Activación

Sigmoid:

- Output entre 0 y 1, útil para probabilidades
- Problema: Vanishing gradient (gradientes muy pequeños en extremos)
- NO recomendada para capas ocultas

Tanh:

- Output entre -1 y 1, centrada en cero
- Mejor que sigmoid pero aún sufre de vanishing gradient

ReLU (Rectified Linear Unit):

- $f(x) = \max(0, x)$ - La MÁS USADA actualmente
- Ventajas: Simple, rápida, no satura en régimen positivo, acelera convergencia
- Problema: Dying ReLU (neuronas que nunca se activan, siempre output 0)

Leaky ReLU:

- $f(x) = \max(0.01x, x)$ - Permite pequeño gradiente negativo
- Soluciona dying ReLU

ELU (Exponential Linear Unit):

- Suaviza valores negativos exponencialmente
- Outputs tienen media cercana a cero

Maxout:

- Generalización de ReLU y Leaky ReLU
- Más parámetros pero más expresiva

4.2 Inicialización de Pesos

- **Inicialización aleatoria pequeña:** Básica pero puede causar problemas
- **Xavier/Glorot Initialization:** Diseñada para sigmoid/tanh, mantiene varianza constante
- **He Initialization:** Óptima para ReLU, tiene en cuenta que ReLU 'mata' mitad de las neuronas

4.3 Batch Normalization

- Normaliza activaciones de cada capa por mini-batch

- Beneficios: Acelera dramáticamente el entrenamiento, permite learning rates más altos, reduce sensibilidad a inicialización, actúa como regularización
- Se aplica antes o después de la función de activación (debate abierto)
- Parámetros aprendibles: γ (escala) y β (desplazamiento)

4.4 Ajuste de Hiperparámetros

- **Grid Search:** Prueba todas las combinaciones de un conjunto de valores
- **Random Search:** Muestra aleatoriamente, a menudo más eficiente que grid search
- **Bayesian Optimization:** Modelo probabilístico para buscar eficientemente

4.5 Transfer Learning

- Reutilizar modelo pre-entrenado en dataset grande (ej: ImageNet) como punto de partida
- Estrategias:
 - **Feature extraction:** Congelar capas convolucionales, entrenar solo capas finales
 - **Fine-tuning:** Entrenar también algunas capas convolucionales con learning rate bajo
- Reduce tiempo de entrenamiento y cantidad de datos necesarios

TEMA 5: CONVOLUTIONAL NEURAL NETWORKS (CNN)

5.1 ¿Por qué CNNs para Imágenes?

- Redes fully connected tienen DEMASIADOS parámetros para imágenes (ej: imagen $224 \times 224 \times 3 = 150,528$ inputs)
- CNNs explotan estructura espacial de imágenes
- Parámetros compartidos → dramática reducción de parámetros

5.2 Capa Convolutiva

- Aplica filtros (kernels) que se deslizan sobre la imagen
- Cada filtro detecta un patrón específico (bordes, texturas, formas)
- Parámetros: tamaño del kernel (típicamente 3×3 , 5×5), stride (desplazamiento), padding
- Produce feature maps que capturan características locales

5.3 Pooling (Agrupación)

- Reduce dimensionalidad espacial manteniendo información importante
- **Max Pooling:** Toma el valor máximo de cada región (el MÁS USADO)
- **Average Pooling:** Toma el promedio de cada región
- Proporciona invarianza a pequeñas traslaciones

5.4 Arquitectura Típica de CNN

CONV → ReLU → POOL → CONV → ReLU → POOL → ... → Flatten → FC → Output

- Capas iniciales: detectan features simples (bordes, colores)
- Capas intermedias: combinan features simples en más complejas (texturas, patrones)
- Capas finales: reconocen objetos completos

5.5 Arquitecturas CNN Famosas

LeNet-5 (1998):

- Una de las primeras CNNs exitosas (reconocimiento de dígitos)
- 7 capas, diseñada para imágenes 32×32

AlexNet (2012):

- Ganó ImageNet 2012 con gran margen, marcó inicio era Deep Learning
- 8 capas (5 conv + 3 FC), 60 millones de parámetros
- Innovaciones: ReLU, Dropout, Data Augmentation, entrenamiento en GPUs

VGG (2014):

- Arquitectura muy profunda (16-19 capas) usando solo kernels 3×3

- Simple pero efectiva, aún usada como feature extractor
- 138 millones de parámetros (VGG-16)

GoogLeNet/Inception (2014):

- Introduce módulos Inception (convoluciones de diferentes tamaños en paralelo)
- 22 capas pero solo 5 millones de parámetros (mucho más eficiente que VGG)

ResNet (2015):

- Introduce skip connections (conexiones residuales)
- Permite entrenar redes MUY PROFUNDAS (50, 101, 152, hasta 1000 capas)
- Skip connections previenen vanishing gradient
- Una de las arquitecturas MÁS INFLUYENTES

MobileNet:

- Diseñada para dispositivos móviles (eficiente en cómputo)
- Usa depthwise separable convolutions

EfficientNet (2019):

- Escala balanceada de profundidad, ancho y resolución
- Estado del arte en eficiencia (mejor accuracy con menos parámetros)

5.6 Aplicaciones de CNNs

- Clasificación de imágenes
- Detección de objetos (YOLO, Faster R-CNN)
- Segmentación semántica (U-Net, Mask R-CNN)
- Reconocimiento facial
- Análisis médico (detección de tumores, clasificación de rayos X)

TEMA 6: WORD VECTORS (REPRESENTACIONES DE PALABRAS)

6.1 Problema de Representación de Texto

Las computadoras no entienden palabras directamente. Necesitamos representarlas como números para que los modelos puedan procesarlas.

6.2 One-Hot Encoding

- Representación más simple: vector con 1 en posición de la palabra, 0 en el resto
- Problemas: Alta dimensionalidad (tamaño = vocabulario), sparse, NO captura similaridad semántica
- Ejemplo: 'perro' y 'gato' son igual de diferentes que 'perro' y 'avión'

6.3 Word Embeddings (Vectores Densos)

- Representaciones de baja dimensión (típicamente 50-300 dimensiones) y densas
- Capturan relaciones semánticas y sintácticas
- Palabras similares tienen vectores cercanos en el espacio

6.4 Word2Vec

Desarrollado por Google (Mikolov et al., 2013). Dos arquitecturas:

Skip-gram:

- Predice palabras del contexto dada una palabra central
- Mejor para datasets pequeños y palabras raras

CBOW (Continuous Bag of Words):

- Predice palabra central dado el contexto
- Más rápido que skip-gram

Propiedades notables de Word2Vec:

- Captura analogías: king - man + woman $\approx$ queen
- Paris - France + Italy $\approx$ Rome
- walking - walk + swim $\approx$ swimming

6.5 GloVe (Global Vectors)

Desarrollado por Stanford (Pennington et al., 2014)

- Factorización de matriz de co-ocurrencia global de palabras
- Combina beneficios de métodos globales (factorización matricial) y locales (ventana de contexto)
- A menudo supera a Word2Vec en tareas de analogía

6.6 FastText

Desarrollado por Facebook AI

- Extensión de Word2Vec que considera sub-palabras (n-gramas de caracteres)
- Puede generar embeddings para palabras fuera del vocabulario (OOV)
- Útil para idiomas con morfología rica o palabras raras

6.7 Embeddings Contextuales

- Word2Vec/GloVe dan UN SOLO vector por palabra (sin contexto)
- Embeddings contextuales (ELMo, BERT) generan diferentes vectores según el contexto
- Ejemplo: 'banco' en 'banco de peces' vs 'banco financiero'

TEMA 7: RECURRENT NEURAL NETWORKS (RNN)

7.1 ¿Por qué RNNs?

- Redes feedforward no manejan bien datos secuenciales (texto, series temporales, audio)
- RNNs tienen conexiones recurrentes que permiten 'memoria' de información pasada
- Procesa secuencias de longitud variable

7.2 Arquitectura de RNN Básica

- Estado oculto h_t se actualiza en cada paso de tiempo
- $h_t = f(W_{hh} * h_{t-1} + W_{xh} * x_t + b)$
- El mismo conjunto de pesos se comparte en todos los pasos de tiempo

7.3 Problema del Vanishing Gradient

- En secuencias largas, gradientes se vuelven exponencialmente pequeños
- La red 'olvida' información de pasos temporales lejanos
- RNNs vanilla difícilmente capturan dependencias de largo plazo

7.4 LSTM (Long Short-Term Memory)

Desarrollada por Hochreiter y Schmidhuber (1997). Resuelve vanishing gradient.

Componentes clave:

- **Cell state (C_t):** 'Cinta transportadora' que mantiene información a largo plazo
- **Forget gate:** Decide qué información desechar del cell state
- **Input gate:** Decide qué nueva información agregar al cell state
- **Output gate:** Decide qué parte del cell state mostrar como output

LSTM puede aprender dependencias de MUCHO más largo plazo que RNN vanilla.

7.5 GRU (Gated Recurrent Unit)

Variante más simple de LSTM (Cho et al., 2014)

- Solo 2 gates: reset gate y update gate
- Menos parámetros que LSTM → entrena más rápido
- Performance similar a LSTM en muchas tareas

7.6 Bidirectional RNNs

- Procesa secuencia en AMBAS direcciones (forward y backward)
- Captura contexto pasado Y futuro
- Útil cuando toda la secuencia está disponible (ej: traducción, clasificación de texto)

7.7 Aplicaciones de RNNs/LSTMs

- Procesamiento de lenguaje natural (traducción, análisis de sentimientos)
- Generación de texto
- Reconocimiento de voz
- Series temporales (predicción de stocks, clima)
- Descripción automática de imágenes (image captioning)
- Generación de música

7.8 Limitaciones de RNNs

- Procesamiento secuencial NO paralelizable → lento en secuencias largas
- Aún dificultad con dependencias MUY largas
- Transformers (basados en attention) están reemplazando RNNs en muchas tareas de NLP

TEMA 8: AGENTES INTELIGENTES - DEEP REINFORCEMENT LEARNING

8.1 ¿Qué es Reinforcement Learning?

Paradigma de aprendizaje donde un agente aprende a tomar decisiones mediante interacción con un entorno, recibiendo recompensas o penalizaciones.

8.2 Componentes Fundamentales

- **Agente:** El que toma decisiones
- **Entorno:** Mundo con el que interactúa
- **Estado (s):** Situación actual del entorno
- **Acción (a):** Decisión tomada por el agente
- **Recompensa (r):** Feedback del entorno (positivo o negativo)
- **Política (π):** Estrategia del agente, mapea estados $\rightarrow$ acciones

8.3 Objetivo

Maximizar la recompensa acumulada total (retorno) a largo plazo, no solo la recompensa inmediata.

8.4 Exploración vs Explotación

- **Exploración:** Probar acciones nuevas para descubrir mejores estrategias
- **Explotación:** Usar el conocimiento actual para maximizar recompensa
- Dilema fundamental: equilibrar ambas para aprender eficientemente

8.5 Q-Learning

Algoritmo clásico de RL que aprende una función $Q(s,a)$ que estima la recompensa futura esperada de tomar acción 'a' en estado 's'.

- Actualización: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
- α = learning rate, γ = factor de descuento (importancia de recompensas futuras)

8.6 Deep Q-Network (DQN)

Desarrollado por DeepMind (Mnih et al., 2015)

- Combina Q-Learning con Deep Learning
- Usa una red neuronal para aproximar $Q(s,a)$ en lugar de tabla
- Permite manejar espacios de estados ENORMES (ej: píxeles de videojuegos)

Innovaciones clave de DQN:

- **Experience Replay:** Almacena transiciones (s, a, r, s') en memoria, entrena con mini-batches aleatorios $\rightarrow$ rompe correlación temporal

- **Target Network:** Usa una red separada (actualizada periódicamente) para calcular targets → estabiliza entrenamiento

8.7 Policy Gradients

- Enfoque alternativo a DQN
- Aprende directamente la política $\pi(a|s)$ en lugar de la función de valor Q
- Puede manejar espacios de acciones continuos
- Algoritmos: REINFORCE, Actor-Critic, PPO, A3C

8.8 Actor-Critic

- Combina value-based (critic) y policy-based (actor)
- Actor: aprende la política
- Critic: evalúa la acción tomada
- Reduce varianza de policy gradients

8.9 Casos de Éxito

- **AlphaGo (2016):** Venció al campeón mundial de Go combinando Deep RL, Monte Carlo Tree Search y CNNs
- **AlphaZero:** Aprende desde cero (sin conocimiento humano) ajedrez, Go, shogi en horas
- **DQN jugando Atari:** Superó humanos en 49 juegos usando solo píxeles como input
- **OpenAI Five:** RL para jugar Dota 2
- **Robótica:** Control de robots, manipulación de objetos

TEMA 9: REDES NEURONALES EN ENTORNOS BIG DATA

9.1 Desafíos del Big Data en Deep Learning

- Datasets de terabytes o petabytes que NO caben en memoria de una máquina
- Tiempo de entrenamiento prohibitivo en un solo device
- Necesidad de procesamiento distribuido y paralelo

9.2 Procesamiento Distribuido

- Dividir datos y/o modelo entre múltiples máquinas/GPUs
- Coordinar cómputo para entrenar eficientemente
- Reducir comunicación entre nodos (bottleneck)

9.3 Apache Spark

- Framework de procesamiento distribuido en memoria
- Más rápido que Hadoop MapReduce (100x en memoria, 10x en disco)
- APIs en Scala, Java, Python, R
- Componentes: Spark SQL, MLlib (machine learning), GraphX, Streaming

9.4 Data Parallelism (Paralelismo de Datos)

- Cada worker tiene una COPIA COMPLETA del modelo
- Los datos se dividen entre workers
- Cada worker calcula gradientes en su subset de datos
- Gradientes se agregan (promedio o suma) para actualizar el modelo
- Estrategia MÁS COMÚN

Sincronización:

- **Síncrona:** Espera a que todos los workers terminen antes de actualizar. Más estable pero puede ser lenta si un worker es lento
- **Asíncrona:** Workers actualizan modelo independientemente. Más rápida pero puede ser inestable

9.5 Model Parallelism (Paralelismo de Modelo)

- El modelo se DIVIDE entre workers
- Útil cuando el modelo es TAN GRANDE que no cabe en un solo device
- Más complejo de implementar que data parallelism
- Mayor comunicación entre workers (más overhead)

9.6 Frameworks para Deep Learning Distribuido

Horovod:

- Desarrollado por Uber para entrenar en clusters
- Simplifica data parallelism en TensorFlow, Keras, PyTorch, MXNet
- Usa MPI y NCCL para comunicación eficiente

TensorFlowOnSpark:

- Integra TensorFlow con Apache Spark
- Permite entrenar modelos TF en clusters Spark

Ray:

- Framework para aplicaciones distribuidas y paralelas
- Incluye bibliotecas para RL (RLlib) y hyperparameter tuning (Tune)

9.7 Consideraciones Prácticas

- Batch size: En data parallelism, batch size efectivo = $\text{batch_size_por_worker} \times \text{num_workers}$
- Learning rate: Puede necesitar ajuste al escalar (linear scaling rule)
- Comunicación: Usar InfiniBand o GPUs con NVLink para reducir latencia
- Gradient compression: Reducir bits de gradientes para ahorrar ancho de banda

TEMA 10: ECOSISTEMAS EN LA NUBE Y PUESTA EN PRODUCCIÓN

10.1 Ciclo de Vida de un Proyecto de ML

1. Definición del problema y objetivos
2. Recolección y exploración de datos
3. Preparación de datos (limpieza, feature engineering)
4. Entrenamiento y experimentación de modelos
5. Evaluación y validación
6. Deployment (puesta en producción)
7. Monitoreo y mantenimiento

10.2 MLOps (Machine Learning Operations)

- Conjunto de prácticas para colaboración entre científicos de datos y operaciones
- Automatiza y estandariza el ciclo de vida de ML
- Similar a DevOps pero para modelos de ML

Componentes clave:

- **Versionado:** Código, datos, modelos (DVC, Git LFS)
- **CI/CD:** Integración y deployment continuo de modelos
- **Monitoreo:** Performance en producción, data drift, model drift
- **Reproducibilidad:** Asegurar que experimentos sean replicables

10.3 Containerización con Docker

- Empaqueta aplicación + dependencias en un contenedor portable
- Garantiza que corre igual en desarrollo, testing y producción
- Dockerfile define la imagen del contenedor
- Docker Compose para múltiples contenedores

10.4 Orquestación con Kubernetes

- Automatiza deployment, scaling y management de contenedores
- Auto-scaling basado en carga
- Self-healing: reinicia contenedores que fallan
- Load balancing automático

10.5 Serving de Modelos

TensorFlow Serving:

- Sistema flexible y de alto rendimiento para servir modelos TensorFlow
- Expone modelo vía REST API o gRPC
- Soporte para versionado de modelos

TorchServe:

- Equivalente para PyTorch
- APIs REST y gRPC
- Métricas y logging integrados

ONNX Runtime:

- Formato abierto para modelos de ML (ONNX = Open Neural Network Exchange)
- Permite convertir modelos entre frameworks (PyTorch → TensorFlow, etc.)
- Optimizado para inferencia rápida

10.6 Plataformas Cloud para ML

AWS (Amazon Web Services):

- **SageMaker:** Plataforma completa para build, train, deploy modelos
- **EC2 P3/P4 instances:** VMs con GPUs NVIDIA V100/A100
- **S3:** Almacenamiento escalable para datasets

Google Cloud Platform (GCP):

- **AI Platform:** Entrenamiento y deployment de modelos TensorFlow, scikit-learn, XGBoost
- **TPUs (Tensor Processing Units):** Hardware especializado de Google para acelerar entrenamiento
- **Vertex AI:** Suite unificada de herramientas ML

Azure (Microsoft):

- **Azure Machine Learning:** Workspace para experimentación y deployment
- **Azure Databricks:** Plataforma basada en Apache Spark

10.7 Monitoreo en Producción

- **Model drift:** Degradación de performance porque datos de producción cambian vs datos de entrenamiento
- **Data drift:** Cambio en distribución de features de entrada
- **Métricas a monitorear:** Latencia, throughput, accuracy, precisión, recall, F1
- **Alertas:** Notificaciones cuando métricas caen bajo umbrales
- **Re-entrenamiento:** Actualizar modelo periódicamente con datos nuevos

TEMA 11: ÚLTIMOS AVANCES EN APRENDIZAJE PROFUNDO

11.1 Transformers

Paper fundacional: 'Attention is All You Need' (Vaswani et al., 2017)

Características clave:

- Basado COMPLETAMENTE en mecanismo de attention (sin recurrencia)
- Permite paralelización total (a diferencia de RNNs)
- Captura dependencias de largo alcance eficientemente
- Arquitectura encoder-decoder

Self-Attention:

- Cada palabra 'atiende' a todas las demás en la secuencia
- Calcula pesos de atención basados en similitud (Query, Key, Value)
- Multi-head attention usa múltiples representaciones en paralelo

11.2 BERT (Bidirectional Encoder Representations from Transformers)

Desarrollado por Google (Devlin et al., 2018)

- Pre-entrena de manera BIDIRECCIONAL (contexto izquierdo Y derecho)
- Tareas de pre-entrenamiento:
 - **Masked Language Model (MLM)**: Predice palabras enmascaradas aleatoriamente
 - **Next Sentence Prediction (NSP)**: Predice si dos oraciones son consecutivas
- Fine-tuning en tareas específicas (clasificación, NER, QA)
- Revolucionó NLP alcanzando estado del arte en 11 tareas

11.3 GPT (Generative Pre-trained Transformer)

Desarrollado por OpenAI

GPT-1 (2018):

- Decoder Transformer autoregresivo (predice siguiente palabra)
- Pre-entrena en grandes corpus, fine-tune en tareas específicas

GPT-2 (2019):

- 1.5 billones de parámetros
- Genera texto muy coherente
- Zero-shot learning: resuelve tareas SIN fine-tuning

GPT-3 (2020):

- 175 billones de parámetros (modelo MÁS GRANDE en su momento)
- Few-shot learning: aprende con pocos ejemplos (5-10) en el prompt
- Capacidades emergentes: traducción, código, razonamiento, matemáticas

ChatGPT (2022) y GPT-4 (2023):

- Entrenado con Reinforcement Learning from Human Feedback (RLHF)
- Conversacional, sigue instrucciones, rechaza contenido inapropiado
- GPT-4: multimodal (texto + imágenes)

11.4 GANs (Generative Adversarial Networks)

Propuestas por Ian Goodfellow (2014)

Arquitectura:

- **Generator (G):** Genera datos falsos intentando engañar al discriminador
- **Discriminator (D):** Distingue entre datos reales y falsos
- Entrenamiento adversarial: juego minimax entre G y D

Aplicaciones:

- Generación de imágenes realistas (rostros, paisajes)
- Style transfer (transferencia de estilo)
- Image-to-image translation (CycleGAN, pix2pix)
- Super-resolution (mejorar resolución de imágenes)
- Data augmentation (generar datos sintéticos)

Variantes famosas:

- **DCGAN:** Usa CNNs
- **StyleGAN:** Control detallado sobre estilo de imágenes generadas
- **BigGAN:** Genera imágenes de alta resolución

11.5 Vision Transformers (ViT)

Aplica arquitectura Transformer a imágenes (Dosovitskiy et al., 2020)

- Divide imagen en patches (ej: 16x16 píxeles)
- Trata patches como 'palabras' en una secuencia
- Rivaliza o supera CNNs en clasificación de imágenes
- Requiere MÁS DATOS que CNNs para entrenar desde cero

11.6 Self-Supervised Learning

- Aprende representaciones útiles SIN etiquetas humanas

- Crea tareas supervisadas automáticamente a partir de los datos
- Ejemplos: predecir rotación de imagen, colorizar imágenes en escala de grises, MLM en BERT
- **SimCLR, MoCo**: Métodos contrastivos para aprendizaje visual

11.7 Few-Shot y Zero-Shot Learning

- **Few-Shot**: Aprender con muy pocos ejemplos (1-10)
- **Zero-Shot**: Resolver tareas SIN ejemplos de entrenamiento específicos
- Modelos grandes como GPT-3 exhiben estas capacidades

11.8 Modelos Multimodales

- Procesan múltiples tipos de datos (texto, imagen, audio, video)
- **CLIP (OpenAI)**: Une visión y lenguaje, permite zero-shot classification de imágenes
- **DALL-E**: Genera imágenes a partir de descripciones textuales
- **Flamingo, Gato**: Modelos visuales-lingüísticos de DeepMind

11.9 Neural Architecture Search (NAS)

- Automatiza el diseño de arquitecturas de redes neuronales
- Usa RL o evolución para buscar arquitecturas óptimas
- EfficientNet fue descubierta con NAS
- Costoso computacionalmente pero puede descubrir arquitecturas superiores

RESUMEN EJECUTIVO

Este manual cubre los fundamentos y técnicas avanzadas de Sistemas Cognitivos Artificiales, desde los conceptos básicos de Deep Learning hasta los últimos avances en la investigación. Los temas principales incluyen:

- **Fundamentos teóricos:** Redes neuronales, backpropagation, funciones de activación, optimización
- **Frameworks modernos:** TensorFlow, Keras, PyTorch para implementación eficiente
- **Técnicas avanzadas:** Regularización, Batch Normalization, Transfer Learning
- **Arquitecturas especializadas:** CNNs para visión, RNNs/LSTMs para secuencias
- **Representaciones de texto:** Word2Vec, GloVe, embeddings contextuales
- **Aprendizaje por refuerzo:** DQN, Policy Gradients, aplicaciones en juegos y robótica
- **Escalabilidad:** Procesamiento distribuido con Spark, data/model parallelism
- **Producción:** MLOps, containerización, serving de modelos, monitoreo
- **Estado del arte:** Transformers, BERT, GPT, GANs, Vision Transformers, modelos multimodales

El campo de Deep Learning evoluciona rápidamente, con nuevos avances que constantemente amplían las capacidades de los sistemas de inteligencia artificial. El conocimiento de estos conceptos fundamentales y técnicas avanzadas es esencial para cualquier profesional que trabaje en el desarrollo de sistemas cognitivos artificiales.